

CMPT 225-D100 Quiz 4, Summer 2026

Duration	30 minutes for the entire exam.
Coding Questions	For coding questions, use C++-like pseudocode. This means that small coding errors will usually be overlooked. However, it must be clear that the code could easily be converted to real C++.
Aids allowed	Pencil or pen, and eraser. No notes, no papers, no books, no computers, no calculators, no cheat sheets, etc.
During the exam	Raise your hand if you would like to speak with a proctor. To be fair to all students, we don't answer questions about the exam. Choose the answer based on your best understanding of the

question. Re-reading the question may help.

1. Write an O-notation expression of the form " $O(f(n))$ " that describes the **worst-case running time** of these algorithms for sorting n items. Assume there are no duplicates and that nothing is known about the initial order of the items.

	Insertion sort	Heap sort	Merge sort	Quicksort (regular)	Quicksort (randomized)	Bucket sort	Radix sort
Worst case running time	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n^2)$	$O(n + N)$ N is max # among keys; 0.5 marks if $O(n)$	$O(n)$ assuming fixed digit-length of keys; or $O(d(n + N))$

2. Prove that $\log n! \leq n \log n$ for all integers $n > 0$.

We can write $\log n!$ as $\log n! = \log 1 + \log 2 + \dots + \log n$. Since each of those n individual terms is $\leq \log n$, we get:

$$\log n! = \log 1 + \log 2 + \dots + \log n \leq \log n + \log n + \dots + \log n = n \log n$$

3. Assuming p points to the root of a **binary search tree (BST)**, write a function called `contains(p, x)` that efficiently returns true if x is a key in the tree rooted at p , and false otherwise. **Use C++-like pseudocode**, and assume the usual BST node structure.

```
bool contains(Node* p, int k) {  
    if (p == nullptr) {  
        return false;  
    } else if (p->key == k) {  
        return true;  
    } else if (k < p->key) {  
        return contains(p->left, k);  
    } else {  
        return contains(p->right, k);  
    }  
}
```

4. State the **height-balance property for AVL trees**.

For every internal node v of tree T , the heights of the children of v differ by at most 1.

5. Draw the **AVL tree** that results after inserting 35 into this:

